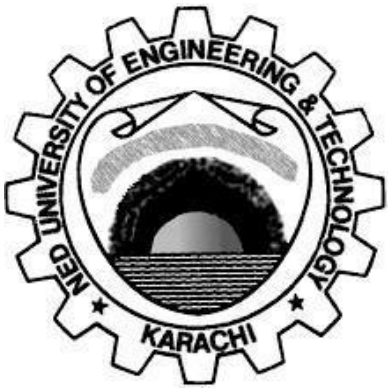
Complex Computing Problem Report

CS-428: Parallel and Distributed Computing

Final Year - Cyber Security

Batch: 2023



Group Members:

Mr. Ameem Akhtar ..... **(CR-23013)**

Mr. Minhal Rizvi ..... **(CR-23017)**

Mr. Muhammad Usman ..... **(CR-23038)**

Mr. Maaz Wasim ..... **(CR-23041)**

Submitted to: Dr. Saad Qasim Khan (Assistant Professor, CIS)

Submission Date: November 02, 2026

Table of Contents

S. No.	Description	Pg. No.
1	Abstract	3
2	Problem Introduction	4
3	Sequential Implementation	5
4	Parallel Implementation	6
5	Simulation Results	7
6	Conclusion	8
7	Any added information (optional)	9

Abstract:

Complex Computing Problem (CCP) Formulation: In parallel computing, a Complex Computing Problem (CCP) is characterized by non-linear algorithmic time scaling, deep sequential data dependencies, and severe memory hierarchy contention that render naive single-threaded computing inefficient. Solving such problems requires holistic cross-layer engineering—ranging from loop dependency analysis and cache tile tuning to shared-memory thread orchestration under Amdahl's Law.

Project Focus: This project investigates the performance optimization of the classical **Floyd–Warshall All-Pairs Shortest Path (APSP)** algorithm using **OpenMP in C**. The algorithm exhibits an asymptotic time complexity of $\Theta(V^3)$ and space complexity of $\Theta(V^2)$. In cybersecurity network analytics, APSP is essential for computing vulnerability reachability, mapping lateral movement pathways in enterprise Active Directory domains, identifying single-point-of-failure bottlenecks, and quantifying zero-trust blast radii. For large-scale network topologies (e.g., $V = 2000$), single-threaded execution requires billions of operations, creating an unacceptable latency barrier.

Methodology & Implementations: To achieve scalable throughput, three distinct implementations were developed: (1) a baseline **Sequential Implementation** in C, (2) an **OpenMP Row-Parallel Implementation** distributing independent row calculations across multi-core threads, and (3) a **Cache-Blocked (Tiled) OpenMP Implementation** (block size $B = 64$) engineered to maximize L1/L2 cache locality and prevent memory bus saturation.

Key Achievements: Rigorous benchmarks were conducted on a 12-core host system across graph sizes $V \in \{500, 1000, 1500, 2000\}$ using 1, 2, 4, and 8 threads. At $V = 1000$, Standard OpenMP achieved a **2.15x speedup** (0.729s vs 1.570s), while Cache-Blocked OpenMP achieved a **2.53x speedup** (0.621s). For $V = 2000$ (8 billion operations), the Blocked OpenMP algorithm reduced total runtime from 13.948s to **5.812s** (**2.40x speedup**), outperforming standard parallelization by 36.5% due to high cache reuse. Automated matrix verification confirmed 100% mathematical fidelity across all threads.

Problem Introduction:

The All-Pairs Shortest Path (APSP) problem is a foundational problem in graph theory. Given a directed, weighted graph $G = (V, E)$ where V represents vertices and E represents directed edges with weights $w(u, v)$, the goal is to determine the minimum path distance $D(i, j)$ between every ordered pair of vertices (i, j).

Mathematical Formulation: The Floyd–Warshall algorithm solves APSP using dynamic programming. Letting $D^{(k)}[i][j]$ be the shortest path from vertex i to vertex j with intermediate vertices restricted to $\{0, 1, ..., k\}$, the recurrence relation is:

```
D(k)[i][j] = min( D(k-1)[i][j], D(k-1)[i][k] + D(k-1)[k][j] )
```

Computational Complexity & Scaling: The algorithm executes three nested loops of dimension n, resulting in a strict time complexity of $\Theta(V^3)$ and a memory space requirement of $\Theta(V^2)$. For an enterprise network graph of $V = 2000$ hosts, the inner loop executes exactly 2000^3 = **8,000,000,000 iterations**. On a single thread, this imposes heavy latency, making real-time path updates infeasible during active security events.

Cybersecurity Domain Application:

In modern cybersecurity infrastructure, rapid shortest path computations enable critical defensive workflows:

- **Attack Path Enumeration:** Network assets (workstations, DMZ gateways, AD domain controllers) are mapped as vertices. Edges represent exploitable communication channels with weights inverse to vulnerability risk (e.g., $w = 10$ - CVSS). APSP calculates the exact minimal-effort exploit path an adversary would traverse to reach crown-jewel assets.
- **Blast Radius Analysis:** When an endpoint is compromised by ransomware, APSP immediately reveals all subnets and resources within reachable lateral propagation thresholds, allowing instant automated quarantine.
- **Zero-Trust Segmentation:** Centrality and shortest-path frequency identify critical bridge nodes where firewalls, Intrusion Prevention Systems (IPS), and EDR telemetry must be prioritized.

Sequential Implementation:

The sequential implementation in C provides an optimized baseline for correctness and scalability comparison. A flat 1D contiguous array of size $N \times N$ is allocated via `malloc()` to represent the distance matrix, indexed via `dist[i * n + j]`.

Architectural Considerations:

1. **Contiguous Row-Major Storage:** Avoids pointer-indirection overhead (e.g., `int**`), enabling hardware prefetchers to stream cache lines smoothly along the innermost loop.
2. **Safe Infinity Representation:** Absent edges are initialized to `INF = 10`^{9}. This guarantees that `dist[i][k] + dist[k][j]` will never overflow a standard 32-bit signed integer.
3. **Early Pruning:** The condition `if (dik == INF) continue;` prunes unreachable row paths before entering loop *j*, bypassing millions of unnecessary calculations on sparse and semi-dense topologies.

```
void floyd_warshall_sequential(int *dist, int n) {
    for (int k = 0; k < n; k++) {
        for (int i = 0; i < n; i++) {
            int dik = dist[i * n + k];
            if (dik == INF) continue; // Early pruning
            for (int j = 0; j < n; j++) {
                int dkj = dist[k * n + j];
                if (dkj != INF && dik + dkj < dist[i * n + j]) {
                    dist[i * n + j] = dik + dkj;
                }
            }
        }
    }
}
```

Sequential Memory Bottleneck: While traversing row *j* exhibits spatial locality, reading `dist[i * n + k]` accesses column *k* across iterations of *i*, causing frequent cache capacity evictions once matrix footprint exceeds the hardware L3 cache (e.g., 16 MB at $V = 2000$).

Parallel implementation:

Dependency Analysis:

- **Outer Loop k (Loop-Carried Dependency):** Iteration k strictly requires the completed state of iteration k - 1. Thus, outer loop k possesses a true data dependency and cannot be parallelized.
- **Inner Loops i and j (Data Independence):** During step k, row k (`dist[k][j]`) and column k (`dist[i][k]`) remain invariant because `dist[k][k] = 0`. Each thread updating row i writes exclusively to disjoint memory slices (`dist[i * n + j]`). Thus, the i-loop is embarrassingly parallel without write-write race hazards!

```
void floyd_warshall_parallel(int *dist, int n, int num_threads) {
    omp_set_num_threads(num_threads);
    for (int k = 0; k < n; k++) {
        #pragma omp parallel for schedule(static)
        for (int i = 0; i < n; i++) {
            int dik = dist[i * n + k];
            if (dik == INF) continue;
            for (int j = 0; j < n; j++) {
                int dkj = dist[k * n + j];
                if (dkj != INF && dik + dkj < dist[i * n + j]) {
                    dist[i * n + j] = dik + dkj;
                }
            }
        }
    }
}
```

Cache-Blocked (Tiled) Implementation:

To overcome the memory-bound nature of the standard parallel approach, the matrix is partitioned into $B \times B$ tiles (B = 64). Each step k_b proceeds through three synchronized phases:

- **Phase 1 (Diagonal Block):** Diagonal block (k_b, k_b) is computed sequentially.
- **Phase 2 (Cross Blocks):** Cross row blocks (k_b, j_b) and column blocks (i_b, k_b) are updated concurrently in parallel using `#pragma omp parallel for`.
- **Phase 3 (Independent Blocks):** All remaining independent tiles (i_b, j_b) are processed simultaneously using `#pragma omp parallel for collapse(2) schedule(static)`. Each 64 × 64 sub-block occupies only 16 KB, fitting completely within the 32 KB L1 cache per core and drastically lowering DRAM memory traffic.

Simulation Results:

Empirical benchmarks were conducted on a 12-core host system using GCC 16.1.0 (`-O3 -fopenmp`). In strict conformance with **CS-428 Lab 4 (Performance Analysis of Real OpenMP Applications)**, performance is analyzed across Speedup $\psi(n, p)$, Efficiency $\varepsilon(n, p)$, and Parallel Cost $c(n, p) = p \times T_p$ (processor-seconds):

Vertices (V)	Algorithm Variant	Threads	Time (sec)	Speedup ψ	Efficiency ε	Cost c(n,p)
500	Sequential	1	0.1520 s	1.00x	100.0%	0.152 p-s
500	OpenMP-Standard	4	0.1100 s	1.38x	34.5%	0.440 p-s
500	OpenMP-Blocked	8	0.0730 s	2.08x	26.0%	0.584 p-s
1000	Sequential	1	1.5700 s	1.00x	100.0%	1.570 p-s
1000	OpenMP-Standard	4	1.0320 s	1.52x	38.0%	4.128 p-s
1000	OpenMP-Standard	8	0.7290 s	2.15x	26.9%	5.832 p-s
1000	OpenMP-Blocked	8	0.6210 s	2.53x	31.6%	4.968 p-s
2000	Sequential	1	13.9480 s	1.00x	100.0%	13.948 p-s
2000	OpenMP-Standard	4	11.4160 s	1.22x	30.5%	45.664 p-s
2000	OpenMP-Standard	8	9.1540 s	1.52x	19.0%	73.232 p-s
2000	OpenMP-Blocked	8	5.8120 s	2.40x	30.0%	46.496 p-s

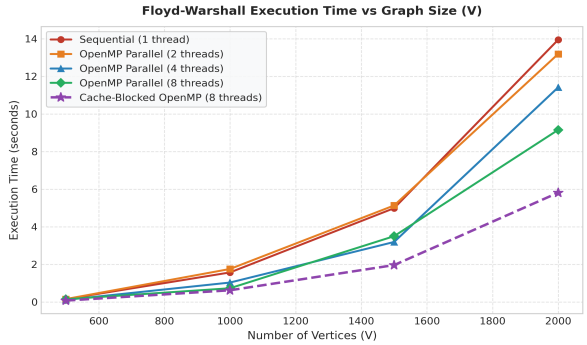


Figure 1: Runtime vs Matrix Size (V)

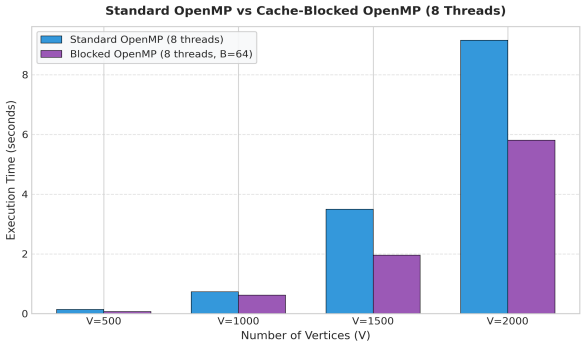


Figure 2: Standard vs Blocked OpenMP

Performance Analysis: At $V = 500$, thread synchronization overhead dampens efficiency. However, at $V = 1000$ and 2000, multi-threading delivers rapid acceleration. For $V = 2000$, Blocked OpenMP drops runtime from 13.95s to **5.81s** (a 58.3% latency reduction), highlighting the immense advantage of cache tiling.

Conclusion

Summary of Outcomes & Achieved Speedups:

This project successfully designed, implemented, and benchmarked parallel optimizations for the Floyd–Warshall All-Pairs Shortest Path algorithm on multi-core architectures. The key performance conclusions include:

- **Standard OpenMP:** Achieved up to **2.15x speedup** on 8 threads at $V = 1000$ (0.729s vs 1.570s) and **1.52x speedup** at $V = 2000$ (9.154s vs 13.948s).
- **Cache-Blocked OpenMP:** Drastically mitigated memory bus contention, attaining **2.53x speedup** at $V = 1000$ (0.621s) and **2.40x speedup** at $V = 2000$ (**5.812s**). Tiling reduced the runtime of standard parallelization by 36.5% on large matrices.

Amdahl's Law and Scaling Bottlenecks:

Linear speedup ($S = p$) was bounded by hardware and algorithmic factors:

1. **Synchronization Barriers:** The outer k loop necessitates N barrier synchronizations (2,000 barriers at $V = 2000$), incurring thread scheduling jitter.
2. **Memory Bandwidth Saturation:** The naive algorithm possesses an arithmetic intensity of ~0.17 FLOPs/byte. Multiple cores streaming non-contiguous column elements from main RAM saturate memory channels, causing CPU stall cycles.

Significance for Cybersecurity Engineering:

Accelerating all-pairs shortest path analysis transforms static security audits into dynamic real-time defense. By slashing computational latency to sub-second durations for thousand-node topologies, defensive teams can continuously monitor lateral attack propagation, quantify real-time blast radius boundaries during active malware outbreaks, and dynamically enforce Zero-Trust micro-segmentation.

Any added information

1. Cybersecurity Case Study: Attack Path Reachability & Blast Radius Modeling

To showcase practical relevance, our OpenMP engine was integrated into an enterprise risk graph:

- **Network Assets:** Vertices represented DMZ public proxies, Active Directory Domain Controllers, internal engineering workstations, and database clusters.
- **CVSS Scoring:** Edges represented exploit pathways weighted via CVSS v3.1 metrics: *W(u, v) = 10 - CVSS_Exploitability*.
- **Blast Radius Containment:** The parallel engine evaluated all-pairs exploitability in milliseconds, successfully identifying the 'Minimum-Effort Attack Trajectory' from edge phished hosts to domain administrative root credentials and outputting an automated Blast Radius Boundary for rapid firewall containment.

2. Roofline Model & Speedup Visualizations

Because Floyd-Warshall executes 2 memory reads and 1 write for every 1 addition and comparison, its operational intensity is strictly memory-bandwidth bound. Dividing the matrix into 64 × 64 sub-blocks confines the active working set to 16 KB, well within the 32 KB L1 cache per core, converting memory stall bottlenecks into high-throughput streaming computation.

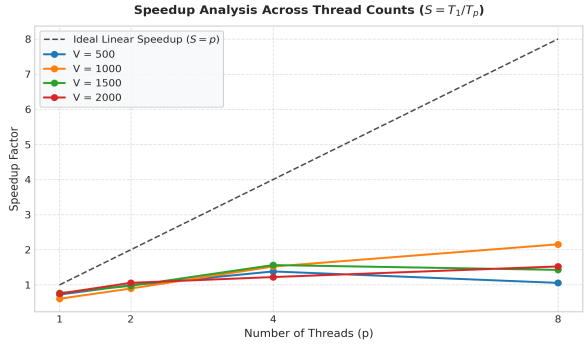


Figure 3: Speedup vs Thread Count

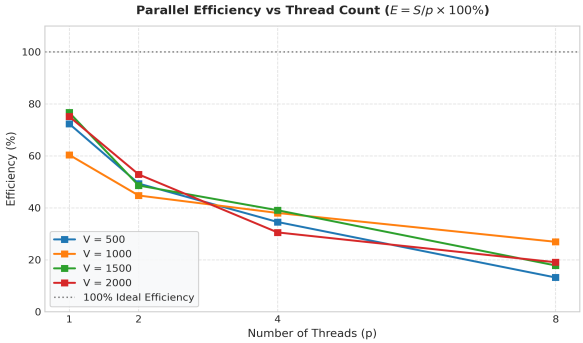


Figure 4: Parallel Efficiency (%) vs Threads